Python E-Course

Module 7 — File Handling

Detailed Notes

Module Overview

Level: Intermediate

Duration: ~1 week

Prerequisites: Modules 1-6

What You'll Learn

- Open, read, and write text files.
- Use context managers (with).
- Work with CSV and JSON files.
- Navigate the filesystem with pathlib.

Lessons

#	Lesson	Duration
1	Opening Files & Modes	30 min
2	Reading and Writing Text	30 min
3	Context Managers (with)	25 min
4	CSV and JSON	40 min
5	Paths with pathlib	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 8: Error Handling.

Lesson 1: Opening Files & Modes

Module: 7 — File Handling

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Open a file with open().
- Choose the right mode: r, w, a, x, b, +.
- Always close files (and why context managers solve this).

Prerequisites

- Module 6, especially encoding (L5).

1. Why This Matters

Almost every program reads or writes a file — config, logs, data, exports. Doing it wrong can corrupt files or leak file descriptors. The rules are simple; learn them once and they apply everywhere.

2. Concept

2.1 open() signature

```
file = open(path, mode="r", encoding="utf-8")
```

Returns a file object you can read from or write to.

2.2 Modes

Mode	Meaning
r	Read (default). File must exist.
w	Write. Overwrites existing file or creates new.
a	Append. Adds to end; creates if missing.
x	Exclusive create. Fails if file exists.
b	Binary mode (add to others: rb, wb).
+	Read AND write (rarely used directly).
t	Text (default).

Default is rt — read text.

2.3 Always specify encoding

```
open("data.txt", "r", encoding="utf-8")
```

Without encoding=, Python uses the OS default — UTF-8 on macOS/Linux but cp1252 on Windows. That's a bug waiting to happen.

2.4 Closing files

```
f = open("a.txt")
data = f.read()
f.close()
```

If you forget close(), the file may not flush to disk and you leak resources. Use with instead (Lesson 3) — it closes automatically.

2.5 Binary mode

For non-text files (images, zip files):

```
with open("image.jpg", "rb") as f:
    data = f.read()        # bytes, not str
```

2.6 Text mode handles newlines

In text mode on Windows, Python automatically converts \r\n to \n on read and back on write. In binary mode, no conversion happens.

3. Code Examples

Example 1: Open and close (with try/finally for safety)

```
f = open("hello.txt", "w", encoding="utf-8")
try:
    f.write("Hello, file!\n")
finally:
    f.close()
print("done")
```

Expected Output: done (and a hello.txt file is created)

Explanation: try/finally guarantees close, even if write fails.

Example 2: Write modes — w overwrites

```
open("note.txt", "w", encoding="utf-8").write("v1\n")
open("note.txt", "w", encoding="utf-8").write("v2\n")
# After both writes, note.txt contains only 'v2\n'
print("After 'w' twice, file has only the last write.")
```

Expected Output: After 'w' twice, file has only the last write.

Example 3: Append

```
open("log.txt", "w", encoding="utf-8").write("line 1\n")
open("log.txt", "a", encoding="utf-8").write("line 2\n")
open("log.txt", "a", encoding="utf-8").write("line 3\n")
print(open("log.txt", encoding="utf-8").read())
```

Expected Output:

```
line 1
line 2
```

```
line 3
```

Example 4: Exclusive create

```
import os
if os.path.exists("once.txt"):
    os.remove("once.txt")
open("once.txt", "x", encoding="utf-8").write("first time\n")
try:
    open("once.txt", "x", encoding="utf-8")
except FileExistsError as e:
    print("Caught:", e)
```

Expected Output: Caught: [Errno 17] File exists: 'once.txt'

Example 5: Binary mode

```
data = "café".encode("utf-8")
with open("b.bin", "wb") as f:
    f.write(data)

with open("b.bin", "rb") as f:
    raw = f.read()
print(raw)
print(raw.decode("utf-8"))
```

Expected Output:

```
b'caf\xc3\xa9'
café
```

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting close()	Data may not flush; file locked	Use with (next lessons)
Opening w on a file you wanted to read	Overwrites everything	Use r
Not specifying encoding	OS-default surprise	Always encoding="utf-8"
Reading binary as text	Decoding error	Use rb

5. Practice Tasks

- Open greet.txt in w mode, write three lines, close. Verify with a text editor.
- Repeat with a mode and check the file grew.
- Try opening a non-existent file in r mode and observe the FileNotFoundError.

6. Key Takeaways

- open(path, mode, encoding=...) is the entry point.
- Modes: r read, w write (overwrite), a append, x create-or-fail, b binary.
- Always specify encoding="utf-8" for text files.

- Always close — best done with with (Lesson 3).

7. What's Next

How to actually read and write the contents — by character, line, or as one big string.

Lesson 2: Reading and Writing Text

Module: 7 — File Handling

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Read a whole file, line-by-line, or in chunks.
- Write strings and lists of strings.
- Understand newline handling.

Prerequisites

- Module 7 Lesson 1

1. Why This Matters

There are multiple ways to read a file — some load the whole thing into memory (fine for small files, terrible for a 5 GB log). Knowing which to use is the difference between code that scales and code that crashes.

2. Concept

2.1 Reading methods

Method	Returns
f.read()	Entire file as one string
f.read(n)	Next n characters
f.readline()	Next line (including \n)
f.readlines()	All lines as a list
for line in f:	Iterate line-by-line — best for large files

The iteration form is memory-friendly: only one line in memory at a time.

2.2 Writing methods

Method	What it does
f.write(s)	Write a string
f.writelines(iterable)	Write each item; does NOT add newlines

To write a list of lines with newlines:

```
f.writelines(line + "\n" for line in lines)
```

2.3 Newlines

Each print() ends with \n. Each f.write() writes exactly what you give it. So f.write("hi") then f.write("there") produces hithere, not two lines. You must include \n yourself.

2.4 Reading patterns

Whole file (small):

```
text = open("a.txt", encoding="utf-8").read()
```

Line-by-line (large):

```
with open("a.txt", encoding="utf-8") as f:
    for line in f:
        process(line.rstrip("\n"))
```

As a list:

```
lines = open("a.txt", encoding="utf-8").read().splitlines()
```

3. Code Examples

Example 1: Write then read whole

```
with open("note.txt", "w", encoding="utf-8") as f:
    f.write("hello\n")
    f.write("world\n")

with open("note.txt", encoding="utf-8") as f:
    print(f.read())
```

Expected Output:

```
hello
world
```

Example 2: readlines

```
with open("note.txt", encoding="utf-8") as f:
    lines = f.readlines()
print(lines)
```

Expected Output: ['hello\n', 'world\n']

Explanation: Each element keeps its trailing newline.

Example 3: Iterate line-by-line

```
with open("note.txt", encoding="utf-8") as f:
    for line in f:
        print(repr(line))
```

Expected Output:

```
'hello\n'
'world\n'
```

Example 4: splitlines (cleanest)

```
with open("note.txt", encoding="utf-8") as f:
    lines = f.read().splitlines()
print(lines)
```

Expected Output: ['hello', 'world']

Explanation: splitlines() strips trailing newlines.

Example 5: writelines

```
items = ["apple", "banana", "cherry"]
with open("fruits.txt", "w", encoding="utf-8") as f:
    f.writelines(item + "\n" for item in items)

print(open("fruits.txt", encoding="utf-8").read())
```

Expected Output:

```
apple
banana
cherry
```

4. Common Mistakes

Mistake	What Happens	Fix
writelines(list) without \n	All run together	Add \n per item
Reading huge file with .read()	Out of memory	Iterate for line in f:
Forgetting rstrip("\n") when processing lines	Trailing newlines in your data	.rstrip("\n") or splitlines()
Mixing read and write on same handle	Confused state	Open separate handles, or use mode r+ carefully

5. Practice Tasks

- Write five lines to numbers.txt using a loop and f.write.
- Read it back as a list of stripped lines.
- Count the number of lines and characters in a file you create.

6. Key Takeaways

- read() for small files; iterate for line in f: for big ones.
- write() writes exactly what you give it — include \n.
- splitlines() is the cleanest way to get a list of newline-free lines.

7. What's Next

Context managers — the with statement that makes all of this safer.

Lesson 3: Context Managers (with)

Module: 7 — File Handling

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use with open(...) as f: to guarantee files close.
- Explain why context managers are idiomatic in Python.
- Handle multiple files in one with.

Prerequisites

- Module 7 Lesson 2

1. Why This Matters

If your code raises an exception between open() and close(), the file may never close — leaking handles. with solves this with a single tidy syntax that always cleans up.

2. Concept

2.1 The with statement

```
with open("a.txt", encoding="utf-8") as f:
    data = f.read()
# file is automatically closed here, even if read() raises
```

When the block ends — normally or via exception — the file is closed.

2.2 Multiple files

Use a comma:

```
with open("in.txt", encoding="utf-8") as src, \
     open("out.txt", "w", encoding="utf-8") as dst:
    dst.write(src.read().upper())
```

Or use parens in 3.10+:

```
with (
    open("in.txt", encoding="utf-8") as src,
    open("out.txt", "w", encoding="utf-8") as dst,
):
    ...
```

2.3 Why context managers exist

The pattern "acquire a resource, do work, release the resource — even on errors" comes up everywhere: files, network sockets, locks, database connections. with makes that pattern one line.

We won't write our own context managers in this module — that comes later — but you'll be a consumer of many.

2.4 Equivalent try/finally

with is a shortcut for:

```
f = open("a.txt", encoding="utf-8")
try:
    data = f.read()
finally:
    f.close()
```

Same behavior; much less code.

3. Code Examples

Example 1: Basic with

```
with open("hello.txt", "w", encoding="utf-8") as f:
    f.write("Hello, with!\n")

with open("hello.txt", encoding="utf-8") as f:
    print(f.read())
```

Expected Output: Hello, with!

Example 2: Counting lines

```
with open("hello.txt", encoding="utf-8") as f:
    n = sum(1 for _ in f)
print(f"Lines: {n}")
```

Expected Output: Lines: 1

Explanation: Iterates lines, counts them, never loads the whole file into memory.

Example 3: Copy a file

```
with open("hello.txt", encoding="utf-8") as src, \
     open("hello_copy.txt", "w", encoding="utf-8") as dst:
    for line in src:
        dst.write(line)
print("Copied.")
```

Expected Output: Copied.

Example 4: Even if an exception happens, file still closes

```
try:
    with open("hello.txt", encoding="utf-8") as f:
        data = f.read()
        raise RuntimeError("boom")
except RuntimeError as e:
    print("Caught:", e)
print("File was closed automatically.")
```

Expected Output:

```
Caught: boom
File was closed automatically.
```

4. Common Mistakes

Mistake	What Happens	Fix
Using f after with block ends	ValueError: I/O operation on closed file	Work inside the with block
Manually closing inside with	Double-close (harmless but ugly)	Trust the with
Nested open without with	Resource leak risk	Always with

5. Practice Tasks

- Rewrite the Lesson-1 try/finally example using with.
- Copy a small text file to a new file using with.
- Open a file, read it, raise an intentional ValueError inside the block, catch it outside, and confirm the file is closed (its .closed attribute is True).

6. Key Takeaways

- with open(...) as f: is the idiomatic way to open files.
- The file closes automatically when the block ends.
- Use multiple files with a comma; group with parens in 3.10+.

7. What's Next

Real-world data formats: CSV and JSON.

Lesson 4: CSV and JSON

Module: 7 — File Handling

Duration: 40 minutes

Difficulty: Intermediate

Learning Objectives

- Read and write CSV files using csv.reader, csv.writer, and DictReader/DictWriter.
- Read and write JSON files using json.load/json.dump.
- Convert between Python dicts/lists and JSON.

Prerequisites

- Module 7 Lesson 3, Module 5 Lesson 7

1. Why This Matters

Most real data on disk is CSV (spreadsheet exports, log dumps) or JSON (APIs, configs). Python has first-class stdlib modules for both — no extra installs. You'll use these constantly.

2. Concept

2.1 CSV — reader

```
import csv

with open("data.csv", encoding="utf-8") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)        # list of strings
```

Each row is a list of strings. CSV doesn't have types; numbers are still strings.

2.2 CSV — writer

```
import csv

with open("out.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "age"])
    writer.writerow(["Alice", 30])
```

Always use newline="" when opening a CSV file for writing — without it, Windows adds extra blank lines.

2.3 CSV — DictReader / DictWriter

Treat each row as a dict keyed by the header:

```
import csv

with open("data.csv", encoding="utf-8") as f:
```

```
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])
```

Write:

```
with open("out.csv", "w", encoding="utf-8", newline="") as f:
    fields = ["name", "age"]
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerow({"name": "Alice", "age": 30})
```

2.4 JSON — load and dump

```
import json

data = {"name": "Alice", "scores": [90, 85, 92]}

with open("user.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

with open("user.json", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded)
```

2.5 String round-trips

```
json_text = json.dumps(data)         # dict -> str
parsed = json.loads(json_text)       # str -> dict
```

Note: dump/load (file), dumps/loads (string).

2.6 What JSON supports

Python	JSON
dict	object {}
list, tuple	array []
str	string
int, float	number
True/False	true/false
None	null

Sets, dates, custom classes don't serialize directly — you'll need to convert them first.

2.7 JSON indentation

json.dump(data, f, indent=2) produces human-readable output. Without indent, it's compact (good for APIs, bad for diffs).

3. Code Examples

Example 1: Write a CSV

```
import csv
```

```
rows = [
    ["Alice", 30, "Bengaluru"],
    ["Bob", 25, "Mumbai"],
    ["Carol", 28, "Delhi"],
]

with open("people.csv", "w", encoding="utf-8", newline="") as f:
    w = csv.writer(f)
    w.writerow(["name", "age", "city"])
    w.writerows(rows)

print(open("people.csv", encoding="utf-8").read())
```

Expected Output:

```
name,age,city
Alice,30,Bengaluru
Bob,25,Mumbai
Carol,28,Delhi
```

Example 2: Read CSV as dicts

```
import csv

with open("people.csv", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['name']} ({row['age']}) from {row['city']}")
```

Expected Output:

```
Alice (30) from Bengaluru
Bob (25) from Mumbai
Carol (28) from Delhi
```

Example 3: Write JSON

```
import json

data = {
    "name": "Maya",
    "skills": ["Python", "SQL"],
    "years": 3,
}
with open("maya.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

print(open("maya.json", encoding="utf-8").read())
```

Expected Output:

```
{
  "name": "Maya",
  "skills": [
    "Python",
```

```
    "SQL"
  ],
  "years": 3
}
```

Example 4: Read JSON

```
import json

with open("maya.json", encoding="utf-8") as f:
    user = json.load(f)
print(user["name"])
print(user["skills"][0])
```

Expected Output:

```
Maya
Python
```

Example 5: JSON string round-trip

```
import json
data = {"a": 1, "b": [2, 3]}
s = json.dumps(data)
print(s)
back = json.loads(s)
print(back == data)
```

Expected Output:

```
{"a": 1, "b": [2, 3]}
True
```

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting newline="" on CSV write	Blank lines between rows on Windows	Always add newline=""
Treating CSV values as ints	They're strings	int(row["age"])
Trying to JSON-serialize a set or datetime	TypeError	Convert to list / iso string first
Confusing dump/dumps	Wrong type argument	dump = file, dumps = string

5. Practice Tasks

- Write 3 students with name, age, score to students.csv. Read back with DictReader and print.
- Build {"city": "Pune", "pin": 411001, "tags": ["IT", "edu"]} and save as JSON with indent 2.
- Load it back and print just the tags.

6. Key Takeaways

- csv and json are in the standard library — no install.
- DictReader/DictWriter are usually friendlier than the list-based readers.
- json.dump/load for files; json.dumps/loads for strings.

- Always newline="" when writing CSV.

7. What's Next

The last lesson — pathlib, the modern way to handle file paths cross-platform.

Lesson 5: Paths with pathlib

Module: 7 — File Handling

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build cross-platform paths with pathlib.Path.
- Read, write, and inspect files via the Path API.
- List, glob, and walk directories.

Prerequisites

- Module 7 Lesson 4

1. Why This Matters

Hard-coded path strings like "data/users.txt" break on Windows if you write "data\\users.txt" and on Mac/Linux if you forget. pathlib gives you one elegant API that just works — it's the modern way to handle paths.

2. Concept

2.1 Importing

```
from pathlib import Path
```

2.2 Building paths

```
p = Path("data") / "users.txt"
print(p)        # data/users.txt (Mac/Linux) or data\users.txt (Windows)
```

The / operator joins path segments — independent of the OS separator.

2.3 Useful properties

```
p = Path("/home/maya/photos/sunset.jpg")
p.name          # 'sunset.jpg'
p.stem          # 'sunset'
p.suffix        # '.jpg'
p.parent        # PosixPath('/home/maya/photos')
p.parts         # ('/', 'home', 'maya', 'photos', 'sunset.jpg')
```

2.4 Read/write shortcuts

```
Path("note.txt").write_text("Hello!", encoding="utf-8")
content = Path("note.txt").read_text(encoding="utf-8")

Path("data.bin").write_bytes(b"\x00\xff")
raw = Path("data.bin").read_bytes()
```

For larger files, use open() to stream — these shortcuts read everything into memory.

2.5 Existence and type checks

```
p = Path("note.txt")
p.exists()     # True/False
p.is_file()
p.is_dir()
```

2.6 Creating and deleting

```
Path("logs").mkdir(exist_ok=True)
Path("logs/2025").mkdir(parents=True, exist_ok=True)
Path("note.txt").unlink(missing_ok=True)   # delete file
```

2.7 Listing and globbing

```
for child in Path(".").iterdir():
    print(child)

for py in Path(".").glob("*.py"):
    print(py)

for py in Path(".").rglob("*.py"):   # recursive
    print(py)
```

2.8 Current and home

```
Path.cwd()       # current working directory
Path.home()      # user home directory
```

2.9 Stat

```
p.stat().st_size      # bytes
p.stat().st_mtime     # last modified (epoch)
```

3. Code Examples

Example 1: Build and inspect a path

```
from pathlib import Path

p = Path("data") / "reports" / "april.csv"
print(p)
print(p.name, p.suffix, p.stem)
print(p.parent)
```

Expected Output (Mac/Linux):

```
data/reports/april.csv
april.csv .csv april
data/reports
```

Example 2: Read and write

```
from pathlib import Path

p = Path("greeting.txt")
p.write_text("Hello, pathlib!\n", encoding="utf-8")
print(p.read_text(encoding="utf-8"))
```

Expected Output: Hello, pathlib!

Example 3: Existence check

```
from pathlib import Path
p = Path("greeting.txt")
print(p.exists())
print(p.is_file())
```

Expected Output:

```
True
True
```

Example 4: Make a folder and write inside

```
from pathlib import Path

Path("output").mkdir(exist_ok=True)
(Path("output") / "log.txt").write_text("line 1\n", encoding="utf-8")
print((Path("output") / "log.txt").read_text(encoding="utf-8"))
```

Expected Output: line 1

Example 5: Glob

```
from pathlib import Path
# create a few .txt files for the demo
for name in ["a.txt", "b.txt", "notes.md"]:
    Path(name).touch()

for f in Path(".").glob("*.txt"):
    print(f)
```

Expected Output (in some order):

```
a.txt
b.txt
```

4. Common Mistakes

Mistake	What Happens	Fix
Building paths with + and os.sep	Works but ugly and error-prone	Use Path / "subpath"
Forgetting parents=True for nested dirs	FileNotFoundError	mkdir(parents=True, exist_ok=True)
Path("...").read_text() on huge file	Out of memory	Stream with open()
Comparing paths as strings	Different separators may not match	Compare Path(a) == Path(b)

5. Practice Tasks

- Build a path Path("notes") / "2025" / "may.md". Print its .parent and .stem.
- Create a folder outputs/ and write hello.txt inside it using pathlib.

- List all .md files in the current folder using glob.

6. Key Takeaways

- from pathlib import Path — modern, cross-platform paths.
- Build with /; read/write with read_text / write_text.
- Use iterdir, glob, rglob for directory traversal.
- mkdir(parents=True, exist_ok=True) is the safe default for creating folders.

7. What's Next

End of Module 7. Next: Module 8 — Error Handling. File operations naturally raise exceptions, so this is the perfect next step.

Module 7 — Exercises

15 exercises.

7.1 open / modes

- (E) Open a.txt for write, write "hi", close. Confirm file exists.
- (M) Append three lines to log.txt using mode "a".
- (H) Attempt to read a missing file; catch and print the FileNotFoundError message.

7.2 read/write

- (E) Write 4 lines, read them back as a list using splitlines().
- (M) Count the lines and characters in a file you create.
- (H) Copy source.txt to dest.txt line-by-line.

7.3 with

- (E) Convert this to with:

```
f = open("x.txt"); data = f.read(); f.close()
```

- (M) Open two files in one with and concatenate their contents into a third.
- (H) Trigger an exception inside with, catch outside, confirm f.closed.

7.4 CSV/JSON

- (E) Write [{name, age}] for 3 people to people.csv using DictWriter.
- (M) Read it back with DictReader and print one person.
- (H) Convert students.csv → students.json (a list of dicts).

7.5 pathlib

- (E) Print Path.cwd() and Path.home().
- (M) Create folder out/data/, then write hello.txt inside it.
- (H) List all .py files under your project recursively with rglob.

Module 7 — Quiz

10 MCQs.

Q1

Which mode overwrites an existing file? A. r B. w C. a D. x

Answer: B (L1)

Q2

Which mode creates only if file doesn't exist? A. r B. w C. a D. x

Answer: D (L1)

Q3

What's the best way to ensure a file is closed? A. Manual close() B. with open(...) as f: C. Leaving it to Python's garbage collector D. try/finally only

Answer: B (L3)

Q4

f.readlines() returns: A. one string B. a list of strings C. an iterator D. a dict

Answer: B (L2)

Q5

For a 5GB log file, best read pattern is: A. .read() B. .readlines() C. for line in f: D. .read(1) loop

Answer: C (L2)

Q6

For CSV writing on Windows, you should open with: A. "wb" B. "w", newline="" C. "w", encoding="cp1252" D. "w", buffering=0

Answer: B (L4)

Q7

Which JSON function reads from a file? A. json.loads(file) B. json.load(file) C. json.read(file) D. json.parse(file)

Answer: B (L4)

Q8

What does Path("a") / "b" produce? A. error B. "a/b" (str) C. a Path("a/b") D. ["a","b"]

Answer: C (L5)

Q9

What's the modern way to read a small text file with pathlib? A. Path(p).read_text(encoding="utf-8") B. open(Path(p)).read() C. Path(p).open().read() D. All work; A is most concise

Answer: D (L5)

Q10

What does json.dump(data, f, indent=2) do? A. Returns a string of data B. Writes pretty-printed JSON to f C. Validates f is JSON D. Loads JSON from f with 2-space tolerance

Answer: B (L4)

Module 7 — Assignment: Log Analyzer

Estimated time: 2 hours

Combines: Module 7 + Modules 4-6

Brief

Read a server access log (one request per line) and produce a JSON report with statistics: total requests, status code counts, top 5 URLs, top 5 IPs.

Provided Input Format

Create access.log with lines like:

```
192.168.1.1 GET /home 200
192.168.1.2 POST /login 401
192.168.1.1 GET /home 200
192.168.1.3 GET /about 404
192.168.1.2 GET /home 200
```

(Whitespace-separated: IP method path status. Generate at least 30 lines for a meaningful test.)

Requirements

Implement these functions:

- parse_line(line) → dict with keys ip, method, path, status (status is int).
- read_log(path) → list of dicts.
- count_by(records, key) → dict mapping value → count for that key.
- top_n(counter, n) → list of (key, count) sorted desc.
- build_report(records) → dict with keys total, by_status, top_paths, top_ips.
- save_report(report, path) → writes JSON with indent=2.

Main: read access.log, build the report, save to report.json, also print to console.

Constraints

- Modules 1-7 only.
- Use with for all file I/O.
- Use pathlib for file paths.
- Encoding: UTF-8.

Expected Output (sample)

```
== Access Report ==
Total requests: 42
By status:
  200: 30
  404: 8
  401: 4
Top 3 paths:
```

```
  /home: 22
  /about: 12
  /login: 8
Top 3 IPs:
  192.168.1.1: 18
  192.168.1.2: 14
  192.168.1.3: 10
Saved report to report.json
```

Grading Rubric

Criterion	Weight
Functions implemented	35%
Correct counts/top-N	25%
JSON saved properly	15%
Uses with + pathlib	15%
Style + docstrings	10%

Submission

Save as assignment_module7.py and commit your access.log and report.json for verification.

Module 7 — Mini Project: Notes Manager

Overview

A CLI tool that stores quick notes as a JSON file. Add, list, search, and delete notes — all persisted between runs.

Concepts Used

- pathlib + json (M7 L4-L5)
- with (M7 L3)
- Dicts and lists (M5)
- Functions (M4)

Features

- add: prompt for title and body, append to notes list with timestamp.
- list: show all notes with id, date, title.
- view ID: show full body of one note.
- delete ID: remove a note.
- search QUERY: case-insensitive search in titles and bodies.
- Notes are persisted in notes.json in the current directory.

Starter Code

```
# Mini Project: Notes Manager
import json
from datetime import datetime
from pathlib import Path

NOTES_FILE = Path("notes.json")

def load():
    if NOTES_FILE.exists():
        with NOTES_FILE.open(encoding="utf-8") as f:
            return json.load(f)
    return []

def save(notes):
    with NOTES_FILE.open("w", encoding="utf-8") as f:
        json.dump(notes, f, indent=2)

def add(notes):
    title = input("Title: ")
    body = input("Body: ")
    note_id = max((n["id"] for n in notes), default=0) + 1
    notes.append({
        "id": note_id,
        "created": datetime.now().isoformat(timespec="seconds"),
        "title": title,
        "body": body,
```

```
    })

def list_all(notes):
    for n in notes:
        print(f"{n['id']:>3} | {n['created']} | {n['title']}")

def view(notes, nid):
    for n in notes:
        if n["id"] == nid:
            print(f"\n# {n['title']}\n({n['created']})\n\n{n['body']}\n")
            return
    print("Not found.")

def delete(notes, nid):
    for i, n in enumerate(notes):
        if n["id"] == nid:
            del notes[i]
            return
    print("Not found.")

def search(notes, q):
    q = q.lower()
    hits = [n for n in notes
            if q in n["title"].lower() or q in n["body"].lower()]
    for n in hits:
        print(f"{n['id']:>3} | {n['title']}")
    if not hits:
        print("(no matches)")

def run():
    notes = load()
    while True:
        cmd = input("\n[add | list | view ID | del ID | find Q | quit] >
").strip().split(maxsplit=1)
        if not cmd:
            continue
        match cmd[0].lower():
            case "add":
                add(notes); save(notes)
            case "list":
                list_all(notes)
            case "view":
                view(notes, int(cmd[1]))
            case "del":
                delete(notes, int(cmd[1])); save(notes)
            case "find":
                search(notes, cmd[1])
            case "quit":
                print("Bye!"); break
            case _:
                print("Unknown")
```

```
if __name__ == "__main__":
    run()
```

Expected Interaction

```
[add | list | view ID | del ID | find Q | quit] > add
Title: Grocery list
Body: bread, milk, eggs
[add | list | view ID | del ID | find Q | quit] > add
Title: Meeting
Body: 3pm with Asha about Q3
[add | list | view ID | del ID | find Q | quit] > list
  1 | 2026-05-23T11:02:00 | Grocery list
  2 | 2026-05-23T11:03:11 | Meeting
[add | list | view ID | del ID | find Q | quit] > view 2

# Meeting
(2026-05-23T11:03:11)

3pm with Asha about Q3

[add | list | view ID | del ID | find Q | quit] > find bread
  1 | Grocery list
[add | list | view ID | del ID | find Q | quit] > quit
Bye!
```

Extension Challenges

- Add tags (a list per note); allow filtering by tag.
- Edit a note by id.
- Export all notes to a single Markdown file.

Grading Rubric

Criterion	Weight
Persists across runs	25%
All commands work	30%
Uses JSON + pathlib	15%
Search is case-insens.	10%
Code organization	10%
Style	10%